Exact frog-based Frechet distance through event generation and polynomially bounded refinement

Jacobus Conradi, Ivor van der Hoog, Eva Rotenberg

arxiv:2512.07728; CC BY 4.0

Editorial scope. Native arxiv.tex119–1074 retained as complete-body1–956. Count restatable shootup theorem only (body294–296); complete main proof and subroutine are in appendix startingbody781. Lossless simplification and integer-kernel material retained as original context and not split into new tasks. The algorithm repairs nonmonotone VE traversals by generating a minimum set of monotonicity events. Its source-specific kinetic ordering/loser tree, heap and bucket structure handles spawn, undertake and join events, and the proof bounds the event work and exhausted row/column-pair obstructions to obtain exact Frechet distance after polynomially many refinements. All detailed appendix arguments are included. Engineering experiments and simplified support estimates are not counted.

Allowed standard prerequisites (editorial).

- Classical continuous Frechet free-space decisions and critical events; cited prior VE-graph refinement framework; standard heaps and tournament trees.. Original precise definitions, prior statements and citations retained. The source-specific spawn/undertake/join events, loser-tree/bucket algorithm, minimal event argument and polynomial termination proof are fully present in appendix Generating monotonicity events.

1 Introduction

The continuous Fréchet distance between two polygonal curves π and σ is classically described using the analogy of a person walking along π and a dog walking along σ, each moving continuously and monotonically. The continuous Fréchet distance $\mathcal{D}_F(\pi, \sigma)$ is the infimum over all such traversals of the maximum distance between them. A discrete analogue is defined similarly by imagining two frogs that hop monotonically along the vertices of π and σ, leading to the discrete Fréchet distance $\mathbb{D}_F(\pi, \sigma)$.

Let $\pi = (p_1, \ldots, p_n)$ and $\sigma = (s_1, \ldots, s_m)$. From an algorithmic and practical perspective, it is possible to decide whether their continuous or discrete distance is less than some input value Δ in an efficient manner. For a given value Δ, deciding whether $\mathbb{D}_F(\pi, \sigma) \leq \Delta$ reduces to determining whether there exists a monotone path in an $n \times m$ vertex-weighted grid where the weight of (p_i, s_j) is $d(p_i, s_j)$. Removing all vertices with weight greater than Δ and

performing a graph search from (p_1, s_1) to (p_n, s_m) takes $O(nm)$ time. The continuous decision problem admits a related construction that we discuss in the preliminaries.

Computing the Fréchet distance and monotonicity events. Computing the *value* of the Fréchet distance is considerably more intricate. The continuous distance is realised by one of three types of events (see Figure 1). Vertex events correspond to distances between pairs of vertices in $\pi \times \sigma$; edge events correspond to distances between a vertex of one curve and an edge of the other; and monotonicity events correspond to triples (e, s, s'), where e is an edge of one curve, s, s' are vertices of the other curve, and the corresponding distance value is realised by the point on e minimising the maximum distance to s and s'. The discrete distance $\mathbb{D}_F(\pi, \sigma)$ is always realised by a vertex event. As there are $O(nm)$ such events one may therefore compute all distances, sort them, and apply the decision procedure, yielding an $O(nm \log nm)$ algorithm (or $O(nm)$ with a simple dynamic program). This version is also easy to make exact, as one can simply square the vertex-weights to preserve rationality of the input. For the continuous setting, monotonicity events introduce two significant difficulties:

- There are $O(n^3 + m^3)$ such events, so no quadratic-time algorithm can enumerate them.
- Edge and monotonicity events involve equations that contain several square root monomials. They therefore offer no straightforward way to avoid numerical imprecision, even if the input is integral. This causes problems, not only for computing the distance value, but even for computing the correct combinatorial traversal that realises the Fréchet distance.

An exact decision procedure for the continuous variant was provided by Bringmann, Künnemann, and Nusser [7, SoCG'19], who use exact geometric predicates to test $\mathcal{D}_F(\pi, \sigma) \leq \Delta$ in $O(nm)$ time. Their implementation allows an approximation of $\mathcal{D}_F(\pi, \sigma)$ to b bits precision via binary search in $O(nm(\log nm + \log b))$ time, but it does not compute the exact value.

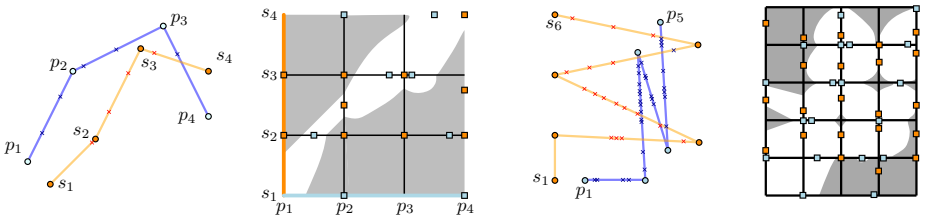


Figure 1: Two curves (π, σ) with their parameter space. White space indicates that the corresponding points are within some distance Δ. In each grid, we illustrate the edge events. For each monotonicity event defined by an edge e and vertices (x_1, x_2) we show the event on e with a cross.

Approaching the continuous Fréchet distance with a frog. Har-Peled, Raichel, and Robson [18, SoCG'25] recently propose a different approach.

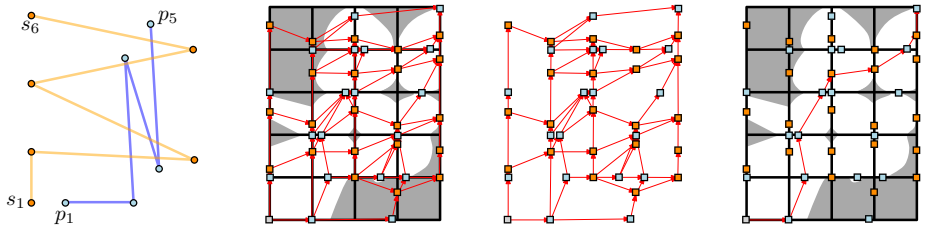


Figure 2: Two pairs of curves (π, σ) and their VE-graph which is a directed and vertex-weighted graph. The minimum-cost path from $(1, 1)$ to (n, m) in this graph is not a monotone traversal.

Given (π, σ) they construct what they call the VE-graph whose vertices correspond to all vertex and edge events. In this graph, there is an edge between two vertices if they share a cell and either the edge is monotone, or the events lie on an edge of the same curve (see Figure 2). The vertices are weighted by the event distance, and the shortest path from $(1, 1)$ to (n, m) in this graph corresponds to a (non-monotone) person-dog traversal γ. If γ happens to be xy-monotone, its continuous counterpart realises the Fréchet distance. Otherwise, non-monotonicities certify that the Fréchet distance is realised by a monotonicity event. Instead of computing monotonicity events, their algorithm bisects each edge of π and σ where γ is non-monotone in the corresponding row or column. This creates higher-complexity curves (π', σ') on which they recurse. They call this *refinement*, and its recursive application yields increasingly complex curves whose vertex and edge events approach the continuous Fréchet distance $\mathcal{D}_F(\pi, \sigma)$. In the limit, this procedure converges to introducing the necessary monotonicity events and thereby computes the continuous Fréchet distance through discrete shortest paths.

Their second contribution is a clever simplification technique that is agnostic to how the Fréchet distance between curves is computed. They show how to simplify two input curves (π, σ) into lower-complexity curves $(\overline{\pi}, \overline{\sigma})$ such that $\mathcal{D}_F(\pi, \sigma) = \mathcal{D}_F(\overline{\pi}, \overline{\sigma})$. The procedure is iterative: they first compute simplified curves $(\overline{\pi}, \overline{\sigma})$ and evaluate $\mathcal{D}_F(\overline{\pi}, \overline{\sigma})$ using any method. This yields a person-dog traversal, from which they generate a witness certifying whether $\mathcal{D}_F(\overline{\pi}, \overline{\sigma}) = \mathcal{D}_F(\pi, \sigma)$ holds. If not, the curves are partially un-simplified, reintroducing complexity until the criterion is satisfied. Based on our observations, this simplification step is likely the main source of the speedy performance of their implementation.

Applications of Fréchet distance. The Fréchet distance has many well-known applications, in particular in the analysis and visualisation of movement data [8,12,21,29]. It is a versatile measure used across a wide range of domains, including handwriting [25], coastlines [22], geographic outlines [14], trajectories of vehicles, animals, and sports players [6, 12, 24, 26], air traffic [5], and protein structures [20]. These typically provide their own Fréchet distance implementations. A recent and particularly active domain is *trajectory clustering*, where the task is to group (sub)curves into clusters such that each cluster has low pairwise Fréchet distance. Several approaches adopt this principle, including the subtrajectory clustering algorithm of Agarwal et al. [1], the map construction

and clustering algorithms of Buchin et al. [8,9], the clustering method of Buchin et al. [10] (implemented in the `MOVETK` library [16]), the clustering algorithm of Conradi and Driemel [13], and the clustering of van der Hoog et al. [27]. In the *vast* majority of these examples (clustering and otherwise), the implementation computes the discrete Fréchet distance, as this is considerably easier to evaluate in practice.

Two of these approaches [8, 9] compute the *semi-weak* Fréchet distance, in which the person and the dog are not required to be monotone within an edge. This variant corresponds to the edge-monotone minimum-cost traversal computed by Har-Peled, Raichel, and Robson [18] before their curve refinement procedure is applied. These examples illustrate the need for efficient and adaptable Fréchet distance implementations, and highlight why the refinement-based approach of Har-Peled, Raichel, and Robson is appealing: it suggests the possibility of computing the exact continuous Fréchet distance through a discrete traversal mechanism that is compatible with existing algorithmic paradigms and interfaces.

Contribution. We re-examine the frog-based framework of Har-Peled, Raichel, and Robson [18], as we revisit three aspects of their contribution.

Contribution 1: exact-value computations. We find the over-arching method conceptually very interesting (and even elegant), however, their algorithm does not compute the exact Fréchet distance. They state that "as the calculation of the optimal Fréchet distance involves distances, impreciseness is unavoidable". In practice, there exist exact-evaluation predicates. In theory, one can even assume a real RAM. Their current approach would remain imprecise under such assumptions:

1. Their implementation inherently uses floating-point arithmetic and thus impreciseness.
2. However, even with exact real-valued arithmetic, their edge bisection does not introduce the monotonicity events that realise the exact Fréchet distance. This refinement converges to the correct value only in the limit, with no bound on the number of required iterations.
3. Their lossless simplification step is theoretically sound and would yield the exact Fréchet distance under real-valued arithmetic. To ensure that the Fréchet distance remains unchanged after simplification, they impose constraints on how the curves may be simplified which can be relaxed.

The original paper briefly acknowledges the possibility of an exact algorithm. Our motivation is to understand to what extent this framework *can* be made exact, and to provide the required components to do so.

1. We explain in detail how to unpack square-root expressions for exact evaluation. This is non-trivial, particularly for monotonicity events and curve simplifications where several square-root terms must be handled simultaneously.
2. We replace their heuristic refinement procedure with a new output-sensitive algorithm that generates the required monotonicity events on

demand. As a consequence, we obtain provable polynomial-time convergence, even under real-valued computations.

3. We strengthen the lossless simplification technique. Whilst the method from [18] works as a black-box given any Fréchet computation algorithm that outputs a traversal, we note that this frog-based paradigm allows for a tighter algorithm and analysis. This leads to an alternative and new algorithm for lossless simplification under the frog-based method.

Contribution 2: exact implementation. We provide an open-source C++ implementation of our adapted algorithm with our exact arithmetic and monotonicity event-generation. This is thereby the first exact implementation. We experientially compare our solution to the one provided in [18]. Unsurprisingly, our median running times increase due to exact computation. Our exact-value representations simply do now allow for vectorization, which comes at performance cost. Somewhat unexpectedly, the worst-case running time drastically improves. We can show instances where the recursive refinement encounters its worst-case behaviour and terminates with an approximation only after many refinement steps. For some data sets, the worst-case behaviour is such that even our average time is an improvement.

Contribution 3: extensive empirical analysis. Our final contribution is the most significant. [18] claims practical performance gains over prior Fréchet distance computation implementations. We observe that the empirical evaluation in [18] is conducted on animal-tracking data, which exhibit two properties: animal trajectories are goal-oriented (which leads to little to no backtracking), and they frequently contain long runs of near-duplicate vertices (as animals stand still for prolonged periods of time). These characteristics favour the lossless simplification step in [18], as they can effectively reduce the input size before the main computation. We perform a broader evaluation by considering additional data sets: un-edited real-world data set from traffic data and adversarial inputs derived from orthogonal vectors. Across all our data sets, we observe a substantially different performance profile than in [18]. In particular, the highly-optimised implementation of Bringmann, Künnemann, and Nusser `BKN` [7] dominates all frog-based implementations in performance (e.g., Figure 3).

At first sight, our empirical analysis may appear unfavourable to our contribution, as it shows that on many practical instances the performance of this new frog-based approach is surpassed by classical techniques. However, we argue for our significance: We are the first exact Fréchet distance implementation. Moreover, the empirical analysis reported at SoCG'25 [18] claimed strong practical performance over [7]. Our broader evaluation provides a complementary picture: it clarifies that this technique is valuable, but also clearly shows the limits of its use. Ultimately, we argue that the frog-based technique of [18] is of genuine theoretical interest, as it can be augmented to compute the exact Fréchet distance in a unique manner with provable guarantees. At the same time, its practical feasibility is debatable.

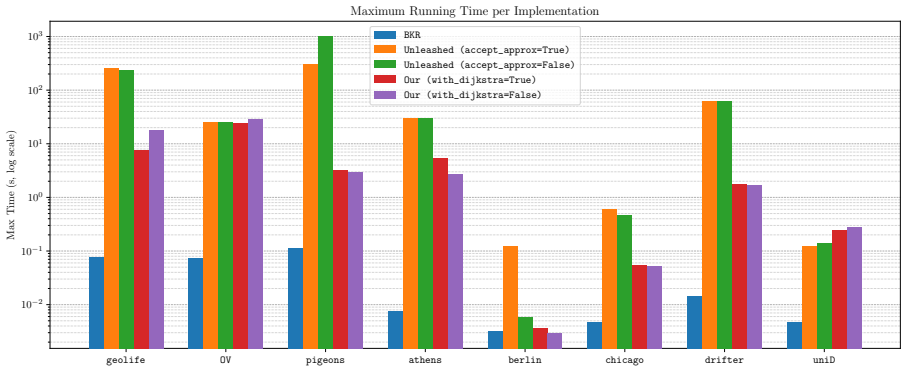


Figure 3: Maximum running times in seconds, log-scale, for curves from eight data sets.

2 Preliminaries

The input consists of polygonal curves π and σ. A *curve* $\pi = (p_1, \ldots, p_n)$ is an ordered sequence of *vertices* connected by straight-line *edges*. We implicitly endow every n-vertex curve π with a bijective, monotone parametrisation $\pi : [1, n] \to \mathbb{R}^d$ such that $\pi(i) = p_i$ for all integers i. For $t \in [i, i+1]$, the mapping is defined by linear interpolation $\pi(t) := p_i + (p_{i+1} - p_i)\,(t - i)$.

For an ordered pair of curves (π, σ) of sizes (n, m), a *traversal* is any pair of continuous, monotone mappings $\alpha : [0, 1] \to [1, n]$ and $\beta : [0, 1] \to [1, m]$. The (continuous) Fréchet distance $\mathcal{D}_F(\pi, \sigma)$ is defined as $\mathcal{D}_F(\pi, \sigma) := \min_{\text{traversals } (\alpha,\beta)} \max_{t\in[0,1]} d\big(\pi(\alpha(t)), \sigma(\beta(t))\big)$.

Equivalently, consider the parameter space $\Gamma_{\pi,\sigma} := [1, n] \times [1, m]$, subdivided into an $n \times m$ grid. A traversal corresponds to any continuous, xy-monotone curve $\gamma(t) = (\alpha(t), \beta(t))$ from $(1, 1)$ to (n, m) in $\Gamma_{\pi,\sigma}$. We therefore freely consider traversals as monotone curves.

Deciding. We can decide whether $\mathcal{D}_F(\pi, \sigma) \leq \Delta$ as follows. For each point $(a, b) \in [1, n] \times [1, m]$, define $\phi(a, b) = 0$ if and only if $d\big(\pi(a), \sigma(b)\big) \leq \Delta$. The area $\{(a, b) \mid \phi(a, b) = 0\}$ is the *free space*. A straightforward DFS over the free space decides in $O(nm)$ time whether the free space contains a monotone curve from $(1, 1)$ to (n, m), and hence whether $\mathcal{D}_F(\pi, \sigma) \leq \Delta$.

Computing. The Fréchet distance is realised by one of three types of *events*:

- *Vertex events*: distances between pairs of vertices in $\pi \times \sigma$.
- *Edge events*: distances between a vertex of one curve and an edge of the other.
- *Monotonicity events*: triples (e, s, s') where e is an edge of one curve and s, s' are vertices of the other, corresponding to minimising the maximum distance to s and s'.

A naive algorithm enumerates all events in $O(n^3 + m^3)$ time, sorts them, and performs a binary search with the decision algorithm to obtain $\mathcal{D}_F(\pi, \sigma)$. Alt and Godau [3] observed that many monotonicity events are irrelevant and achieved an $O(nm \log nm)$-time exact algorithm. Har-Peled and Raichel [17] gave a considerably simpler approach: as Δ increases, connected components of the free space merge, and a *critical value* is the smallest Δ at which a new merge occurs. Each cell has one connected component, implying $O(nm)$ critical values. By comparing adjacent cells, all critical values can be generated in $O(nm \log nm)$ time. The fastest algorithm is by Buchin et al. [11], running in $O\big(n^2 \sqrt{\log n} \, (\log\log n)^{3/2}\big)$ time.

Retractability. For curves (π, σ) and a traversal $\gamma = \{\gamma(t) \mid t \in [0,1]\}$, define $\text{cost}(\gamma) := \max_{(a,b)\in\gamma} d\big(\pi(a), \sigma(b)\big)$. Let $(a, b) \in \gamma$ be a point that attains this maximum. Then γ decomposes into two open curves: γ_1 from $(1, 1)$ to (a, b) and γ_2 from (a, b) to (n, m). Har-Peled, Raichel and Robson [18] call γ *retractable* if γ_1 and γ_2 are minimum-cost monotone curves for their respective endpoints, and if γ_1 and γ_2 are themselves retractable (Figure 4). Computing the Fréchet distance using an adaption of Dijkstra yields a retractable traversal.

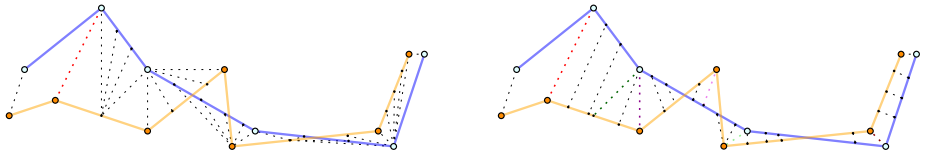


Figure 4: (Left) A traversal between two curves where the red leash realises $\mathcal{D}_F(\pi, \sigma)$. The traversal may be 'lazy': a large Fréchet distance allows long pauses while the other curve progresses. (Right) A retractable traversal achieves progressively smaller bottleneck leash lengths.

3 The frog-based approach

We paraphrase and explain the frog-based approach from [18], defining their VE-graph slightly differently to reduce the number of edge-cases the definition incurs. We then detail how to make it converge in a polynomial number of rounds, even on a real-RAM, by creating an algorithm that generates the required monotonicity events on the fly.

Definition 1. *Each edge e of the grid $\Gamma_{\pi,\sigma}$ has an edge event defining a point that we call an* edge-distance yardstick *(or* eddy*). $V(\pi,\sigma)$ denotes the set of all grid vertices and eddys.*

Definition 2. *The VE-graph is a directed vertex-weighted graph on $V(\pi,\sigma)$. For each cell of $\Gamma_{\pi,\sigma}$, consider its four corners and eddys. We connect all these points in an xy-monotone manner. In addition, the bottom horizontal eddy has a directed edge to the top horizontal eddy, and the left vertical eddy has a directed edge to the right vertical eddy; see Figure 5.*

Definition 3. *A path in the VE-graph is VE-respecting if it runs from $(1,1)$ to (n,m) and, within every row and every column, its restriction to the row or column is connected.*

$$\mathcal{V}_E(\pi,\sigma) := \min_{\text{VE-respecting path } P} \max_{\text{vertex } v\in P} \text{weight}(v). \qquad \text{Observe that } \mathcal{V}_E(\pi,\sigma) \le \mathcal{D}_F(\pi,\sigma).$$

From a path realizing the VE distance, one may also derive an upper bound on $\mathcal{D}_F(\pi,\sigma)$.

Definition 4. *Let P be a path realizing $\mathcal{V}_E(\pi,\sigma)$ then P is a curve. One can greedily morph P into a traversal by ensuring that one never decreases in either coordinate. This defines an xy-monotone traversal $\gamma(t) = (\alpha(t),\beta(t))$. The* Interpolated VE distance *is then defined as:*

$$\mathcal{I}_{VE}(\pi,\sigma) := \max_{t\in[0,1]} d\big(\pi(\alpha(t)),\sigma(\beta(t))\big). \qquad \text{Observe that } \mathcal{D}_F(\pi,\sigma) \le \mathcal{I}_{VE}(\pi,\sigma).$$

The *elevation function* maps $(a,b) \in \Gamma_{\pi,\sigma}$ to $d(\pi(a),\sigma(b))$. The sublevel sets of this function are clipped ellipses, whose minima on each cell edge occur

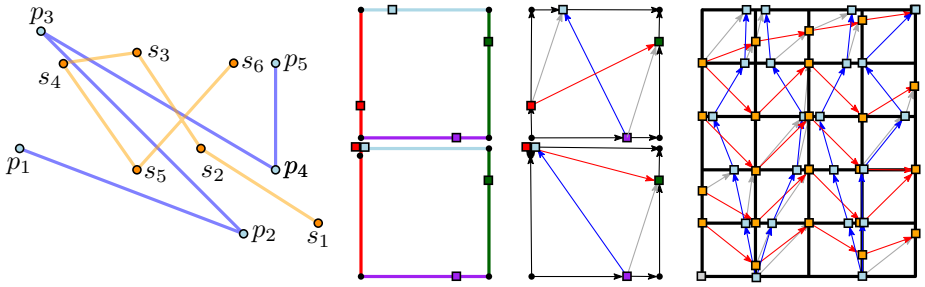


Figure 5: The VE graph considers for each cell the four corners and the four eddys. We include all monotone edges between these vertices. Edges connecting the eddys between horizontal (or vertical) edges may be non-monotone. If vertices coincide, then the picture gets a bit messy.

precisely at the corresponding eddys. It follows that $\mathcal{V}_E(\pi,\sigma) = \mathcal{D}_F(\pi,\sigma) = \mathcal{I}_{VE}(\pi,\sigma)$ whenever the path P realizing $\mathcal{V}_E(\pi,\sigma)$ is monotone. This is the core of the frog-based refinement approach (Algorithm 1).

Algorithm 1 Refinement Procedure from [18]

1: Compute the `VE`-graph, the value $\mathcal{V}_E(\pi,\sigma)$, and the path P realising it
2: **if** P is monotone **then** $\mathcal{V}_E(\pi,\sigma) = \mathcal{D}_F(\pi,\sigma)$ so **return** $\mathcal{V}_E(\pi,\sigma)$
3: Otherwise, insert vertices on π for every non-x-monotone column, and vertices on σ for every non-y-monotone row. Then recurse on the refined curves (π,σ).

3.1 The approach in [18]: bisecting and convergence.

Har-Peled, Raichel, and Robson [18] execute step 3 of Algorithm 1 in a heuristic manner. Let P be a path realising $\mathcal{V}_E(\pi,\sigma)$ for the current input (π,σ). If P is non-monotone in a column that corresponds to an edge e of π then they introduce a vertex half-way e (likewise, they bisect edges of σ for each row where P is non-monotone). Thus, given P they bisect some edges and create higher-complexity curves (π,σ) on which they recurse.

Recall that each monotonicity event (e,p,p') corresponds to a point on the edge e; we refer to such a point as a *monotonicity vertex*. Let (π',σ') be the curves obtained from (π,σ) by inserting, on π and σ respectively, a vertex at every monotonicity vertex. It is observed in [18] that the path P realising $\mathcal{V}_E(\pi',\sigma')$ is xy-monotone, and therefore $\mathcal{V}_E(\pi',\sigma') = \mathcal{D}_F(\pi,\sigma)$. Under exact-value arithmetic, the recursive halving procedure of [18] inserts in the limit all monotonicity vertices of π and σ, and thus converges to the exact Fréchet distance $\mathcal{D}_F(\pi,\sigma)$.

3.2 Our approach and rate of convergence.

In [18] they use floating-point arithmetic. Therefore, edges cannot be bisected in an exact manner and their bisecting never generates the exact monotonicity vertices. They remark that under exact computations one can compute and introduce monotonicity vertices instead, but do not provide details, and naively there are cubicly many such events. Floating-point arithmetic is not unavoidable. Many libraries support exact arithmetic (e.g., CGAL). We engineer a solution that alters step 3 to provide guarantees for exactness and convergence.

Let the path P realising $\mathcal{V}_E(\pi,\sigma)$ be not x-monotone in a column C corresponding to an edge e. Let the subpath of P restricted to C be from a to b. We execute step 3 by computing the minimum number of monotonicity vertices that need to be introduced on e such that, next iteration, the min-cost path from a to b is monotone. Our algorithm is inspired by the critical values from [3, 17] from the preliminaries, but ultimately has a different objective.

On efficiency. Exact-value arithmetic inevitably incurs overhead. Evaluating exact geometric predicates is substantially slower than using floating-point arithmetic. Moreover, the resulting values must be maintained as abstract algebraic objects rather than bit-level representations, preventing efficient vectorisation. This is partly why the algorithm from [7] does not use exact arithmetic

even though it is integrated in CGAL. We therefore expect exact computation to increase median running times when we introduce exact arithmetic. To mitigate this effect, we restrict the input to 32-bit integers, as this allows the most frequent operation—the computation of dot products—to still be efficient. For our exact arithmetic, based on the computed dot products, we discuss in Appendix C our own integer-based kernel, to remain practically somewhat competitive. Importantly, our algorithm does not rely on integrality assumptions: it applies unchanged under any exact-evaluation kernel.

High-level overview. In appendix A we develop the following algorithm: the input is a column C where P is not x-monotone, corresponding to an edge e of π. Let P, restricted to C, go from a to b. We create two new cells by extending a horizontal segment from a and b. We discard all cells of C that do not intersect P. We index the remaining cells as $C_1, \cdot, C_k$ from bottom to top, which induce horizontal segments $h_1, \ldots, h_{k+1}$. Imagine Δ increasing from 0 to ∞. For each value of Δ and each pair (i, j) with $i < j$, h_i can Δ-*reach* h_j if there exists a monotone path from a point on h_i to a point on h_j in the Δ free space (see Figure 6).

A value Δ is an *join event* if there exist (i, j) such that h_i can Δ-reach h_j from Δ onward. In this case, h_i can also Δ-reach all $h_{j'}$ with $j' \in [i, j)$. We compute the minimum number of join events to realise a monotone path from h_1 to h_k using a linked–list of buckets.

The horizontal segments are partitioned into buckets, each forming a vertical interval $[i, j]$ such that h_i can Δ-reach h_j. We initialise Δ as 0, and every horizontal segment forms its own bucket. We then merge buckets in a bottom-up fashion, maintaining the minimum Δ required such that h_1 can Δ-reach h_i for increasing i. Between $(h_1, \ldots, h_i)$ and h_{i+1}, we compute the smallest value $\Delta' \geq \Delta'$ for h_1 can Δ-reach h_{i+1} using a min-heap that contains join events and other events that indicate for which pair $(j, i + 1)$ the join event is relevant.

Whenever we pop the next event, we update Δ. If the updated Δ allows us to Δ-reach h_{i+1} from h_1 then we merge h_{i+1} into the bucket. We then keep incrementing i until h_1 cannot Δ-reach h_{i+1}, at which point we continue the event-based approach. Whenever we merge h_k into the bottom bucket, h_1 can Δ-reach h_k. When all buckets are joined, we have all join events required for a monotone path from h_1 to reach h_k. From this, we obtain the minimum number of monotonicity vertices needed so that, in the next iteration, the min-cost path from a to b is monotone. Doing this for all non-monotone rows and columns implies:

Theorem 5. *Let (π, σ) be the original input curves with n and m vertices. Algorithm 1 can be supported by a data structure using $O(n + m + M)$ space, where $M \in O(n^3 + m^3)$ is the number of introduced monotonicity vertices. Each recursive call runs in $O(nm + M)$ time for Step 1 and $O((n + m + M) \log^2(n + m + M))$ time for Step 3. Under exact-value arithmetic, the procedure returns the exact Fréchet distance after $O(n^3 + m^3)$ recursive calls.*

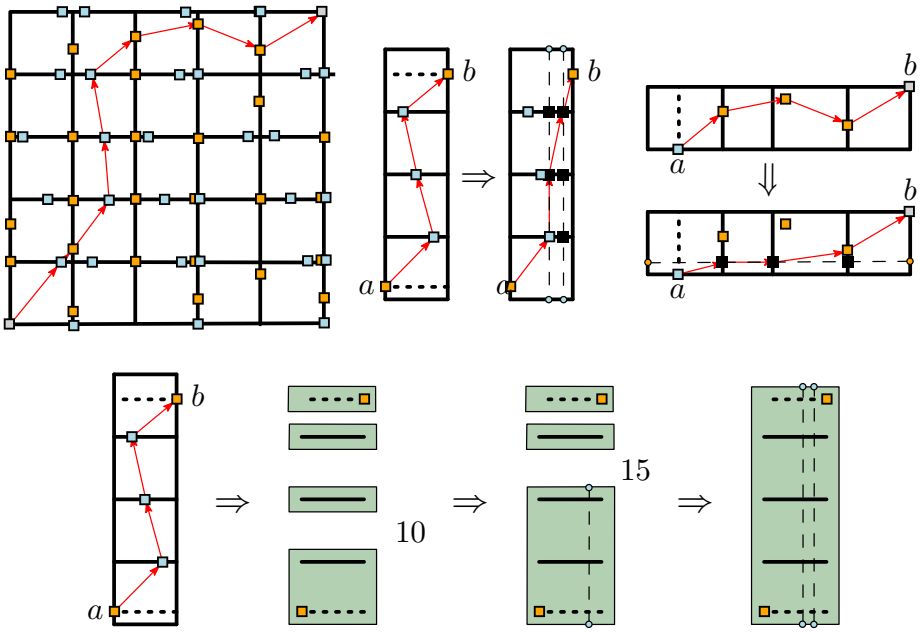


Figure 6: The input is a non-monotone path P in the VE-graph. We identify all rows and columns where P is non-monotone. For a column, we isolate the horizontal segments and bucket them. Between buckets, we store the minimum leash at which a monotone path from bottom to top exists.

4 Lossless Fréchet distance simplification

The second (perhaps even main) contribution of [18] is a black-box lossless simplification algorithm. Computing $\mathcal{D}_F(\pi, \sigma)$ for large complexity curves π and σ is an expensive operation. It would be preferable if we would have access to some lower-complexity curves $(\overline{\pi}, \overline{\sigma})$ where $\mathcal{D}_F(\pi, \sigma) = \mathcal{D}_F(\overline{\pi}, \overline{\sigma})$. Then we could simply operate on these lower-complexity curves to output the distance. Har-Peled, Raichel, and Robson [18] can, for any Fréchet computation algorithm, achieve this using any *vertex-restricted* curve simplification:

Definition 6. *We define for π a (vertex-restricted) simplification $\overline{\pi}$ as any subset of vertices in π, ordered by their index. For any $\mu \in \mathbb{R}_{\geq 0}$ we say $\overline{\pi}$ is a μ-simplification if $\mathcal{D}_F(\pi, \overline{\pi}) \leq \mu$.*

The curve simplification used in [18] is some heuristic combination of techniques from [4, 15].

Naïvely, one can $(1+\varepsilon)$-approximate $\mathcal{D}_F(\pi, \sigma)$ with simplifications by fixing some ε and μ. Compute μ-simplifications $(\overline{\pi}, \overline{\sigma})$ and set $M = \mathcal{D}_F(\overline{\pi}, \overline{\sigma})$. By the triangle inequality, $M - 2\mu \leq \mathcal{D}_F(\pi, \sigma) \leq M + 2\mu$. If $\mu \leq \varepsilon M$, then M $(1+\varepsilon)$-approximates $\mathcal{D}_F(\pi, \sigma)$; otherwise μ is decreased cleverly until we terminate after $O(\log_{1+\varepsilon} n)$ rounds [15].

[18] avoids this approximation step by exploiting triangle inequalities more directly. Let $(\overline{\pi}, \overline{\sigma})$ be any vertex-restricted simplification of (π, σ) and consider the traversals $\gamma_0(t)$, $\gamma_1(t)$, and $\gamma_2(t)$ realising $\mathcal{D}_F(\pi, \overline{\pi})$, $\mathcal{D}_F(\overline{\pi}, \overline{\sigma})$, and $\mathcal{D}_F(\overline{\sigma}, \sigma)$, respectively. They combine these traversals in linear time into a

traversal $\gamma(t) = (\alpha(t), \beta(t))$ between (π, σ) whose maximum leash length gives an upper bound on $\mathcal{D}_F(\pi, \sigma)$. Conversely, $\mathcal{D}_F(\pi, \sigma)$ is at least $\mathcal{D}_F(\overline{\pi}, \overline{\sigma})$ minus the Fréchet distances from each curve to its simplification, and we get the following:

$$\mathcal{D}_F(\overline{\pi}, \overline{\sigma}) - \mathcal{D}_F(\pi, \overline{\pi}) - \mathcal{D}_F(\sigma, \overline{\sigma}) \;\leq\; \mathcal{D}_F(\pi, \sigma) \;\leq\; \max_{t \in [0,1]} \mathcal{D}_F(\pi(\alpha(t)), \sigma(\beta(t))).$$

Computing $\mathcal{D}_F(\pi, \overline{\pi})$ and $\mathcal{D}_F(\sigma, \overline{\sigma})$ is costly, so they compute greedy traversals $\gamma_0^*(t)$ and $\gamma_2^*(t)$ that over-estimate these values instead, yielding $\mathcal{D}_F^*(\pi, \overline{\pi})$ and $\mathcal{D}_F^*(\sigma, \overline{\sigma})$. Combined, these give a greedy traversal $\gamma^*(t) = (\alpha^*(t), \beta^*(t))$ that over-estimates $\mathcal{D}_F(\pi, \sigma)$ and leads to

$$LB(\bar{\pi}, \bar{\sigma}) := \mathcal{D}_F(\bar{\pi}, \bar{\sigma}) - \mathcal{D}_F^*(\pi, \bar{\pi}) - \mathcal{D}_F^*(\sigma, \bar{\sigma}) \leq \mathcal{D}_F(\pi, \sigma) \leq \max_{t \in [0,1]} \mathcal{D}_F(\pi(\alpha^*(t)), \sigma(\beta^*(t))). \tag{1}$$

Introducing slack. Given $LB(\overline{\pi}, \overline{\sigma})$ and $\gamma^*(t)$, each curve vertex v receives a *slack* $\texttt{slack}(v) := LB(\overline{\pi}, \overline{\sigma}) - g_v$, where g_v is the maximum leash length attained at v along $\gamma^*(t)$ (Figure 7). Negative slack marks v as a bottleneck. Each vertex p of $\overline{\pi}$ or $\overline{\sigma}$ inherits the minimum slack of all original vertices between p and the next vertex. Slack is then heuristically propagated: every vertex adopts the minimum slack among its eight closest neighbours along $\overline{\pi}$ or $\overline{\sigma}$. An edge's slack is the minimum slack of its endpoints. They then perform the following:

> while there exists a negative-slack edge e on $\overline{\pi}$ (resp. $\overline{\sigma}$), and the edge e is not an edge of π (resp. σ), un-simplify the edge e (reintroducing missing vertices).

By construction of slack, its propagation, and (1), the process ends with $\mathcal{D}_F(\pi, \sigma) = \mathcal{D}_F(\overline{\pi}, \overline{\sigma})$.

Our contribution. Appendix B shows that the above technique, while elegant, is overly conservative and somewhat coarse. We refine it in two ways.

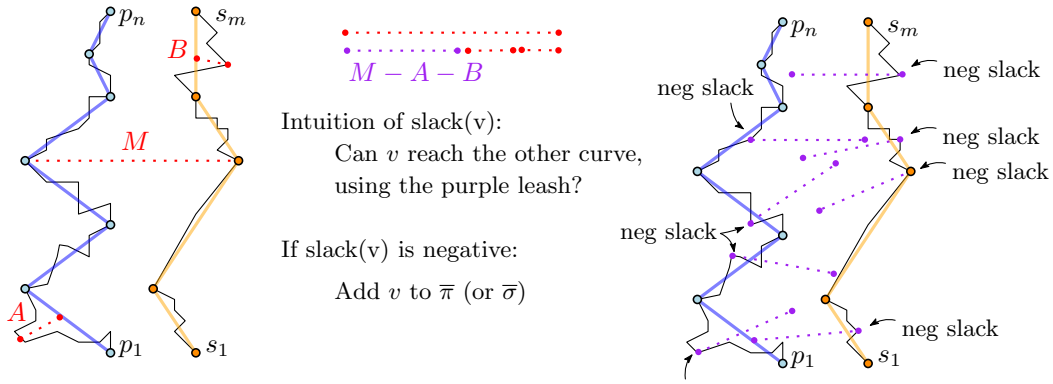


Figure 7: Curves (π, σ) and their simplifications. The value $LB(\overline{\pi}, \overline{\sigma})$ is the maximum leash length M between $(\overline{\pi}, \overline{\sigma})$, minus the maximum leash lengths A and B between $(\pi, \overline{\pi})$ and $(\sigma, \overline{\sigma})$.

(i) Tighter bounds. Their lower bound uses the global maximum leash length M between $(\overline{\pi}, \overline{\sigma})$, minus the global maxima A and B between $(\pi, \overline{\pi})$ and $(\sigma, \overline{\sigma})$, respectively. When M, A and B are realised at unrelated parts of the traversal (see Figure 7), this yields an unnecessarily small value $LB(\overline{\pi}, \overline{\sigma}) = M - A - B$. This in turn causes more vertices than needed to trigger unsimplification. Using the frog-based framework, we obtain a sharper bound (Figure 8). For each edge of $\overline{\pi}$ we compute the maximum leash length between that edge and π, and assign this as a negative weight. In the `VE`-graph, the eddys on this edge add this negative weight to their own weight. The min-cost path in this reweighted graph yields a lower bound for $\mathcal{D}_F(\pi, \sigma)$ by the same triangle-inequality argument as in [18], but now based on local rather than global maxima. For many instances, this increases the lower bound and therefore the slack of vertices — causing our algorithm to unsimplify fewer edges.

(ii) A tighter slack propagation rule. Their propagation step takes the minimum slack over each block of eight consecutive vertices. This removes any edge cases for applying the slack, ensuring their correctness. However, this method is somewhat crude. We tighten this step by propagating slack along at most two edges per vertex, avoiding this coarse eight-neighbour spread.

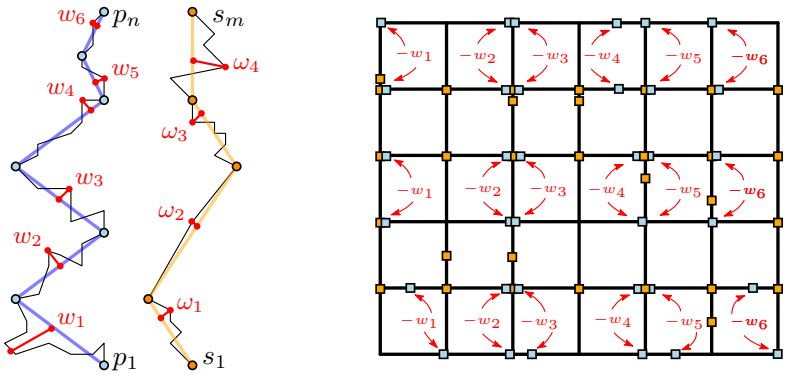


Figure 8: Given $(\pi, \overline{\pi})$ we can give a weight to each edge of $\overline{\pi}$. Each edge over $\overline{\pi}$ is a column in the parameter space, in which we apply its negative weight on the corresponding eddys.

5 Experiments and empirical analysis.

Our primary contribution is a C++ implementation of our exact variant of the frog-based technique, together with an extensive empirical evaluation. We provide two implementations: `Exact_Dijkstra`, which computes the path P in the `VE`-graph using Dijkstra's algorithm in a manner similar to [18] and yields a traversal with retractability properties; and `Exact_Sweepline`, which instead employs a sweepline to compute P, guaranteeing linear space usage. We compare our implementations to `Unleashed`: the Julia code of [18], which is their fastest version. To obtain the most accurate output from `Unleashed`, we invoke `frechet_c_compute` with `accept_approximation=false` (this still approximates). Finally, we include the `BKN` algorithm of [7], the fastest known alternative implementation (which is also inexact). We do not use its CGAL variant, as it is slower, while still being inexact.

Experiments. The analysis in [18] uses three animal-tracking data sets: `Geolife`, `Birds`, and `Pigeons`. These trajectories are goal-oriented. Their implementation benefits from this in one major way compared to [7]: as a simplification is applied, this greatly reduces complexity. Consequently, they operate on significantly smaller effective input.

The lossless simplification of [18] applies to any Fréchet algorithm, including [7]. We *could* enable a direct comparison by applying it to [7]. Yet, we deliberately refrain from doing so: the simplification incurs overhead, and [7] is so fast that this may actually slow it down. Despite not using simplifications, our experiments will show that [7] dominates the frog-based approaches, removing the need to improve it further. All experiments are single-threaded and executed on an AMD Ryzen 5 7600X (4.7 GHz) with 96 GB RAM.

Data. Our real-world data sets are summarised in Tables 1 and 2. The first two (`Geolife`, `Pigeons`) are from [18]. We additionally include `Athens`, `Chicago`, and `Berlin` GPS traffic data sets, widely used in Fréchet benchmarks [2, 8, 19, 28]. We include the `Drifter` data set of [13], containing drifting-buoy trajectories. Finally, we add the UnID data set from [23] which is a traffic dataset captured on German highways. Except for `Drifter`, these sets are goal-oriented with minimal backtracking; lossless simplification therefore removes relatively many vertices, which favours the frog-based approaches. Finally, we include a set `OV`[1] of difficult synthetic pairs of curves that are derived from Orthogonal Vectors instances [7].

The `Geolife` data set alone induces over 170,000,000 pairs of curves, so we cannot possibly compute the Fréchet distance between all pairs of curves. We follow [18] and subsample. In our case, we compare all 45,000 pairs among a subselection of 300 curves from the `Geolife` and `Pigeons` data sets. We further augment the analysis by comparing all curves from the `Athens` data set and from the adversarial `OV` data set. To broaden our evaluation, we also include the remaining real-world data sets, where, due to time constraints, we subsample curves and compare all pairs among them. The experiments are primarily

[1] obtained via personal communication with one author from [7], available together with our source code

bottlenecked by the `Unleashed` implementation[2], and the full evaluation requires several days to complete. Our results indicate no substantial qualitative differences between sampling 100×100 or 300×300 curves from the same data.

Name	Real world	# curves	total # vertices	median # vertices	# of curves compared
`Geolife`	yes	18671	24895648	506	300 choose 2
`Pigeons`	yes	539	6229319	7502	300 choose 2
`Athens`	yes	120	72439	662	120 choose 2
OV	no	220	39060	130	110 (already pairs)
`Chicago`	yes	888	118360	121	100 choose 2
`Berlin`	yes	27188	192223	7	100 choose 2
`Drifter`	yes	2012	1792084	997	100 choose 2
`UniD`	yes	362	214076	495	100 choose 2

Table 1: Overview of all data sets used in our experiments.

Name	Real world	# curves	total # vertices	median # vertices	# of curves compared
`Geolife`	yes	300	396676	729	300 choose 2
`Pigeons`	yes	300	2805495	5411	300 choose 2
`Athens`	yes	120	72439	662	120 choose 2
OV	no	220	39060	130	110 (already pairs)
`Chicago`	yes	100	12731	119	100 choose 2
`Berlin`	yes	100	723	7	100 choose 2
`Drifter`	yes	100	78080	804	100 choose 2
`UniD`	yes	100	55572	421	100 choose 2

Table 2: Overview of all slices of data sets used in our experiments.

Criteria. Our primary evaluation metric is running time (including file-reading time). For exactness, our approach is the most accurate if the input coordinates fit within a 32-bit integer, followed closely by `BKN`; Table 3 briefly reports accuracies for the data sets satisfying this condition. We could compare across all data by truncating all input to 32-bit coordinates, but we refrain from doing so since `BKN` can handle this data natively (albeit inexact).

[2]A time limit of 5 minutes was hit multiple times

Data set	Max	Mean	Median
		`Exact`	
`OV`	0	0	0
`athens`	0	0	0
`UnID`	0	0	0
		`BKN`	
`OV`	$3.039939 \cdot 10^{-13}$	$2.756980 \cdot 10^{-13}$	$3.039939 \cdot 10^{-13}$
`athens`	$9.337306 \cdot 10^{-10}$	$5.865283 \cdot 10^{-12}$	$1.269745 \cdot 10^{-12}$
`UnID`	$9.337306 \cdot 10^{-10}$	$5.865283 \cdot 10^{-12}$	$1.269745 \cdot 10^{-12}$
		`Unleashed`	
`OV`	$4.499666 \cdot 10^{-6}$	$1.430129 \cdot 10^{-6}$	$1.181895 \cdot 10^{-16}$
`athens`	$9.267924 \cdot 10^{-6}$	$1.948025 \cdot 10^{-7}$	$9.974841 \cdot 10^{-15}$
`UnID`	$9.337306 \cdot 10^{-10}$	$5.865283 \cdot 10^{-12}$	$1.269745 \cdot 10^{-12}$

Table 3: Accuracy errors, obtained by converting exact square root distances to a `double`.

5.1 Comparing frog-based approaches.

We first compare our frog-based implementations to `Unleashed`. This comparison is motivated mainly by scientific curiosity: exact-value computation is more difficult than computing an approximation, so one naturally expects a performance penalty. Our interest is in quantifying this penalty—in other words, in understanding the practical cost of exactness. Table 4 compares the efficacy of the `Unleashed` implementation versus `Exact_Sweepline` (the comparison remains roughly the same when we set for `Unleashed` the boolean `accept_approximation=true`, or when we compare to `Exact_Dijkstra` instead).

As expected, the median running time of our exact variants deteriorates. However, we note that our exact approach has considerably better worst-case performance, even on this advantageous real-world data. On some instances, `Unleashed` is a factor 100 slower. Whenever our worst-case is slower, it remains within competitive distance. For data sets where curves have sizable complexity, our method becomes the preferred method from the 90th percentile onwards. On the mapconstruction data sets, the median number of vertices is very small. The OV data set is a difficult instance where our exact computations are extremely costly. We note that except for worst-case outliers, the approaches are relatively competitive. We think that this data nicely represent the cost of exact value computation, and the benefit of worst-case guarantees for runtime.

5.2 Comparing Fréchet distance computations.

We now provide a comparison between our exact frog-based method, the approach of [18], and the state-of-the-art `BKN` implementation [7]. In this wider evaluation, we consider two configurations of `Unleashed`: the version with `accept_approximation=false` (which, still returns an approximation) and the supposedly faster configuration that sets `accept_approximation=true`. Our exact implementations use either Dijkstra or a sweepline.

Table 4: Summary of running times in seconds. For both `Unleashed` (`accept_approx = false`) and `Exact_Sweeline`. The best times across the two methods are in blue.

Data set	Max	Mean	StdDev	Median	80th %ile	90th %ile	95th %ile	99th %ile
				`Unleashed (accept_approx=false)`				
geolife	233.6160	0.3673	3.7389	**0.0069**	**0.0621**	0.3263	1.0108	6.4489
pigeons	>300	0.8508	9.0841	**0.0222**	**0.1645**	0.9395	3.7772	13.5008
athens	29.1741	0.2133	1.3476	**0.0190**	**0.1091**	**0.2808**	**0.5125**	3.6473
OV	**24.6879**	**1.2132**	3.2387	**0.0580**	**1.1886**	**3.7313**	**5.2031**	**15.0121**
chicago	0.4679	**0.0036**	0.0147	**0.0024**	**0.0040**	**0.0053**	**0.0063**	**0.0088**
berlin	0.0059	**0.0001**	0.0001	**0.0001**	**0.0001**	**0.0001**	**0.0001**	**0.0002**
uniD	**0.1402**	**0.0023**	0.0048	**0.0019**	**0.0027**	**0.0032**	**0.0041**	**0.0076**
drifter	61.2818	0.1199	1.1781	**0.0091**	**0.0555**	0.1919	0.4474	1.9376
				`Exact (with_dijkstra=false)`				
geolife	**17.5543**	**0.0984**	0.1903	0.0505	0.1330	**0.2312**	**0.3468**	**0.7444**
pigeons	**2.9708**	**0.1649**	0.1472	0.1265	0.2305	**0.3153**	**0.4130**	**0.7262**
athens	**2.6424**	**0.1617**	0.2359	0.0851	0.2136	0.3816	0.5906	**1.2356**
OV	28.2322	4.5376	6.0343	1.5410	8.5852	13.5812	16.9130	23.8446
chicago	**0.0522**	0.0075	0.0073	0.0043	0.0124	0.0179	0.0231	0.0326
berlin	**0.0029**	0.0004	0.0002	0.0003	0.0004	0.0005	0.0007	0.0014
uniD	0.2809	0.0077	0.0102	0.0064	0.0096	0.0114	0.0139	0.0266
drifter	**1.6604**	**0.0743**	0.0889	0.0505	0.1123	**0.1609**	**0.2215**	**0.4069**

5.3 Results

Table 5 shows the full set of experimental results. These plots highlight the volatility of the frog-based approach: the standard deviation of these approaches is considerably larger than `BKN` (although lessened for our implementation). Regardless of evaluation criterion (maximum, mean or median running time) the `BKN` algorithm dominates in performance[3].

We illustrate the performance profile of these algorithms using Dolan–Moré plots (Figures 9–12). These plots can be read as follows. The x-axis shows a factor $\tau \geq 1$, indicating how many times slower an algorithm may be compared to the best on each instance. The y-axis shows the percentage of instances on which this holds. Thus, if an algorithm's plot contains the point (p, τ) then on p percent of the instances, it is τ times slower than the best performing algorithm. Similarly, $\tau = 1$ reveals how often each algorithm is the best, while a horizontal line such as $p = 90$ indicates how much slower the worst 10% of cases become (which, for `BKN`, is minimal). As τ increases, the curves show how quickly each algorithm "catches up". An algorithm is considered better if its curve approaches the horizontal line $p = 100$ more rapidly. These profiles mirror our observations: `BKN` is consistent; our method lags on median instances, however if the data set contains high enough complexity curves, it frequently and significantly overtakes `Unleashed` as we approach the worst case.

[3]Due to technical constraints, there is system overhead influencing the measured time for `BKN`. This is particularly noticeable for instances of the `berlin` data set since they have so few vertices.

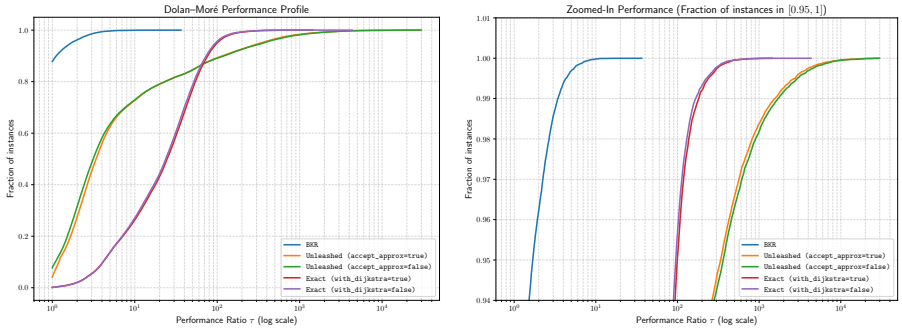


Figure 9: Dolan–Moré performance profile on all pairs of the first 300 curves from the `geolife` data set (44850 in total) from [18].

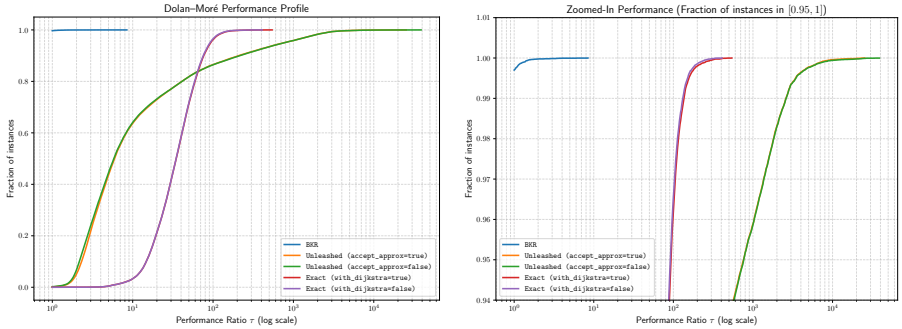


Figure 10: Dolan–Moré performance profile on all pairs of the first 300 curves from the `pigeons` data set (44850 in total) from [18].

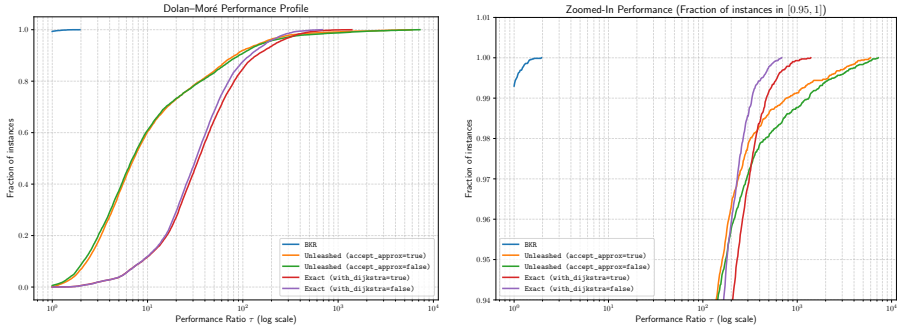


Figure 11: Dolan–Moré profile on pairs from the `athens` data set (7139 in total) from [27].

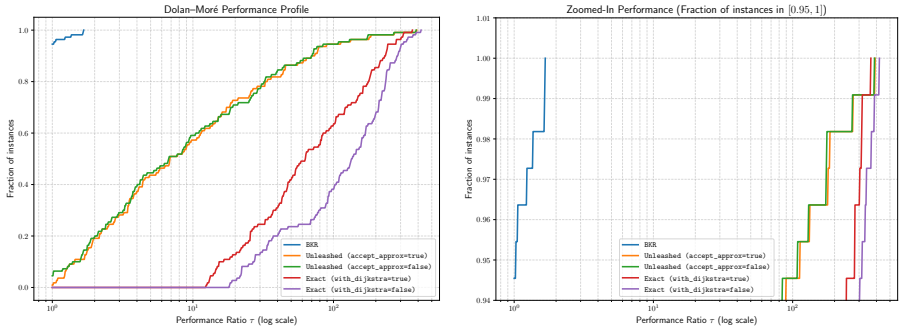


Figure 12: Dolan–Moré performance profile on all pairs from the `OV` data set (110 in total).

6 Conclusion

We studied the frog-based framework for computing the Fréchet distance introduced in [18]. On the theoretical side, we identified three aspects of the original algorithm that incur avoidable inaccuracies in the distance computation or introduce overhead. Specifically, the method of [18] consists of two main components. The first is the computation of the Fréchet distance between (π, σ) via a discrete weighted graph whose complexity is increased iteratively each round, through a process they call *refinement*. Their process heuristically bisects edges of (π, σ), increasing the graph complexity, until the discrete min-cost path through this graph realises the Fréchet distance. We provide a new refinement scheme that offers worst-case guarantees on this process. Our more elaborate refinement is careful not to drastically increase the time per round, yet it yields an explicit upper bound on the convergence rate — which we consider theoretically meaningful.

The second component is a lossless curve simplification procedure that produces curves $(\overline{\pi}, \overline{\sigma})$ of lower complexity such that $\mathcal{D}_F(\pi, \sigma) = \mathcal{D}_F(\overline{\pi}, \overline{\sigma})$ — effectively reducing the input size. Although correct, the original simplification is overly conservative: it relies on a lower bound that is unnecessarily weak, and it avoids geometric corner cases by aggressively propagating vertex slack across a constant but wide neighbourhood. Exploiting the weighted-graph structure already inherent in the frog-based method, we present a more refined construction that attains a higher lower bound. Consequently, our simplifications have lower complexity.

Our primary contribution is an empirical evaluation. We supply an openly available test suite, which enables a reproducible comparison between frog-based implementations and the highly optimised algorithm of Bringmann, Künnemann and Nusser [7]. Our results contrast the findings in [18]: On our broader range of real-world data sets, `BKN` consistently outperforms both frog-based variants. Notably, after removing near-duplicate consecutive points through simplification, the data set curves have *very* few vertices remaining for modern hardware. We have reason to believe that, on larger inputs, the performance gap between our method and `Unleashed` widens. However, we are not aware of real-world benchmark sets whose median complexity exceeds 1,000 "distinct" vertices, and random data behaves entirely differently under lossless simplification. We argue that existing benchmarks are becoming outdated and that the community should develop more modern ones.

At first glance, our findings might seem discouraging: our implementation and analysis describe an approach that does not, in practice, match the performance of the current state-of-the-art. However, we consider this outcome important for the community. Our study offers a transparent and verifiable assessment of the frog-based method and, contrary to earlier claims, demonstrates that its practical behaviour is volatile.

At the same time, our analysis reaffirms that the frog-based framework is theoretically compelling (if you ask us, particularly in its exact variant). We show that this method in particular enables a significantly more fine-grained lossless simplification procedure than general-purpose methods. Nevertheless, our experiments indicate that its main value lies in its theoretical insights rather than in practical performance.

References

[1] Pankaj K Agarwal, Kyle Fox, Kamesh Munagala, Abhinandan Nath, Jiangwei Pan, and Erin Taylor. Subtrajectory clustering: Models and algorithms. *ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems (PODS)*, pages 75–87, 2018. doi:10.1145/3196959.3196972.

[2] Mahmuda Ahmed, Sophia Karagiorgou, Dieter Pfoser, and Carola Wenk. A comparison and evaluation of map construction algorithms using vehicle tracking data. *GeoInformatica*, 19:601–632, 2015. doi:10.1007/s10707-014-0222-6.

[3] Helmut Alt and Michael Godau. Computing the fréchet distance between two polygonal curves. *International Journal of Computational Geometry & Applications*, 5:75–91, 1995. doi:10.1142/S0218195995000064.

[4] Boris Aronov, Sariel Har-Peled, Christian Knauer, Yusu Wang, and Carola Wenk. Fréchet distance for curves, revisited. In *Proceedings of the 14th Annual European Symposium on Algorithms (ESA)*, volume 4168 of *Lecture Notes in Computer Science*, pages 52–63. Springer, 2006. doi:10.1007/11841036_8.

[5] Alessandro Bombelli, Lluis Soler, Eric Trumbauer, and Kenneth D Mease. Strategic air traffic planning with Fréchet distance aggregation and rerouting. *Journal of Guidance, Control, and Dynamics*, 40(5):1117–1129, 2017. doi:10.2514/1.G002308.

[6] Sotiris Brakatsoulas, Dieter Pfoser, Randall Salas, and Carola Wenk. On map-matching vehicle tracking data. In *International Conference on Very Large Data Bases (VLDB)*, pages 853–864, 2005. doi:10.5555/1083592.1083691.

[7] Karl Bringmann, Marvin Künnemann, and André Nusser. Walking the Dog Fast in Practice: Algorithm Engineering of the Fréchet Distance. In *Symposium on Computational Geometry (SoCG)*, volume 129 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 17:1–17:21, Dagstuhl, Germany, 2019. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. URL: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.SoCG.2019.17.

[8] Kevin Buchin, Maike Buchin, David Duran, Brittany Terese Fasy, Roel Jacobs, Vera Sacristan, Rodrigo I Silveira, Frank Staals, and Carola Wenk. Clustering trajectories for map construction. *ACM International Conference on Advances in Geographic Information Systems (SIGSPATIAL)*, pages 1–10, 2017. doi:10.1145/3139958.3139964.

[9] Kevin Buchin, Maike Buchin, Joachim Gudmundsson, Jorren Hendriks, Erfan Hosseini Sereshgi, Vera Sacristán, Rodrigo I Silveira, Jorrick Sleijster, Frank Staals, and Carola Wenk. Improved map construction using subtrajectory clustering. *ACM Workshop on Location-Based Recommendations, Geosocial Networks, and Geoadvertising (SIGSPATIAL)*, pages 1–4, 2020. doi:10.1145/3423334.3431451.

[10] Kevin Buchin, Maike Buchin, Joachim Gudmundsson, Maarten Löffler, and Jun Luo. Detecting commuting patterns by clustering subtrajectories. *International Journal of Computational Geometry & Applications*, 21(03):253–282, 2011. doi:10.1142/S0218195911003638.

[11] Kevin Buchin, Maike Buchin, Wouter Meulemans, and Wolfgang Mulzer. *Four Soviets Walk the Dog—with an Application to Alt's Conjecture*, pages 1399–1413. 2014. doi:10.1137/1.9781611973402.103.

[12] Maike Buchin, Bernhard Kilgus, and Andrea Kölzsch. Group diagrams for representing trajectories. *International Journal of Geographical Information Science*, 34(12):2401–2433, 2020. doi:10.1145/3283207.3283208.

[13] Jacobus Conradi and Anne Driemel. Finding complex patterns in trajectory data via geometric set cover. *arXiv preprint arXiv:2308.14865*, 2023.

[14] Thomas Devogele. A new merging process for data integration based on the discrete Fréchet distance. In *Advances in spatial data handling*, pages 167–181, 2002. doi:10.1007/978-3-642-56094-1_13.

[15] Anne Driemel, Sariel Har-Peled, and Carola Wenk. Approximating the fréchet distance for realistic curves in near linear time. In *International Symposium on Computational Geometry (SoCG)*, pages 365–374. ACM, 2010. doi:10.1145/1810959.1811019.

[16] university of Technology Eindhoven. MoveTK: the movement toolkit. http://https://github.com/movetk/movetk, 2020.

[17] Sariel Har-Peled and Benjamin Raichel. The frechet distance revisited and extended. In *Symposium on Computational Geometry (SoCG)*, page 448–457, New York, NY, USA, 2011. Association for Computing Machinery. doi:10.1145/1998196.1998269.

[18] Sariel Har-Peled, Benjamin Raichel, and Eliot W. Robson. The Fréchet Distance Unleashed: Approximating a Dog with a Frog. In Oswin Aichholzer and Haitao Wang, editors, *International Symposium on Computational Geometry (SoCG)*, volume 332 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 54:1–54:13, Dagstuhl, Germany, 2025. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. doi:10.4230/LIPIcs.SoCG.2025.54.

[19] Jincai Huang, Min Deng, Jianbo Tang, Shuling Hu, Huimin Liu, Sembeto Wariyo, and Jinqiang He. Automatic generation of road maps from low quality gps trajectory data via structure learning. *IEEE Access*, 6:71965–71975, 2018. doi:10.1109/ACCESS.2018.2882581.

[20] Minghui Jiang, Ying Xu, and Binhai Zhu. Protein structure–structure alignment with discrete Fréchet distance. *Journal of bioinformatics and computational biology*, 6(01):51–64, 2008. doi:10.1142/s0219720008003278.

[21] Maximilian Konzack, Thomas McKetterick, Tim Ophelders, Maike Buchin, Luca Giuggioli, Jed Long, Trisalyn Nelson, Michel A Westenberg, and Kevin Buchin. Visual analytics of delays and interaction in movement data.

International Journal of Geographical Information Science, 31(2):320–345, 2017. doi:10.5555/3048386.3048392.

[22] Ariane Mascret, Thomas Devogele, Iwan Le Berre, and Alain Hénaff. Coastline matching process based on the discrete Fréchet distance. In *Progress in Spatial Data Handling*, pages 383–400. Springer, 2006. doi:10.1007/3-540-35589-8_25.

[23] Tobias Moers, Lennart Vater, Robert Krajewski, Julian Bock, Adrian Zlocki, and Lutz Eckstein. The exid dataset: A real-world trajectory dataset of highly interactive highway scenarios in germany. *2022 IEEE Intelligent Vehicles Symposium (IV)*, pages 958–964, 2022. doi:10.1109/IV51971.2022.9827305.

[24] Roniel Sousa, Azzedine Boukerche, and Antonio Loureiro. Vehicle trajectory similarity: Models, methods, and applications. *ACM Computing Surveys*, 53(5), 2020. doi:10.1145/3406096.

[25] E Sriraghavendra, K Karthik, and Chiranjib Bhattacharyya. Fréchet distance based approach for searching online handwritten documents. In *Ninth International Conference on Document Analysis and Recognition (ICDAR)*, pages 461–465, 2007. doi:10.1109/ICDAR.2007.4378752.

[26] Han Su, Shuncheng Liu, Bolong Zheng, Xiaofang Zhou, and Kai Zheng. A survey of trajectory distance measures and performance evaluation. *Proceedings of the VLDB Endowment*, 29(1):3–32, 2020. doi:10.1007/s00778-019-00574-9.

[27] Ivor van der Hoog, Lara Ost, Eva Rotenberg, and Daniel Rutschmann. Efficient Greedy Discrete Subtrajectory Clustering. In *International Symposium on Computational Geometry (SoCG)*, volume 332 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 78:1–78:20, Dagstuhl, Germany, 2025. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. URL: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.SoCG.2025.78.

[28] Suyi Wang, Yusu Wang, and Yanjie Li. Efficient map reconstruction and augmentation via topological methods. *ACM International Conference on Advances in Geographic Information Systems (SIGSPATIAL)*, pages 1–10, 2015. doi:10.1145/2820783.2820833.

[29] Dong Xie, Feifei Li, and Jeff M Phillips. Distributed trajectory similarity search. *Proceedings of the VLDB Endowment*, 10(11):1478–1489, 2017. doi:10.14778/3137628.3137655.

A Generating monotonicity events

Let π and σ be two curves whose original complexity had n and m vertices respectively, where we added M monotonicity vertices to them. Consider their VE-graph and let P be a min-cost path from $(1, 1)$ to (n, m). We can identify each row R and column C in which P is non-monotone. We present a subroutine algorithm, whose input the input is a column C where P is not x-monotone,

corresponding to an edge e of π. Let P, restricted to C, go from a to b. We show how to introduce a minimum-size set of vertices V along e such that, in the resulting denser VE-graph, the minimum cost path from a to b is monotone. We show that we can apply this subroutine to row R and column C in $O((n + m + M) \log^2(n + m + M))$ total time.

A.1 Solving the subproblem.

Given C and a and b, we create two new cells by extending a horizontal segment from a and b. We discard all cells of C that do not intersect P. This creates a new set of cells $C_1, \ldots, C_k$ ordered from bottom to top, where the bottom left of C_1 equals a and the top right of C_k equals b. This, in turn, induce horizontal segments. Consider some value $\Delta \geq 0$. We define the Δ free space as all points (x, y) for which $d(\pi(x), \sigma(y)) \leq \Delta$. We denote this area as Δ-FSD and the horizontal segments induce intervals $h_1, \ldots, h_{k+1}$ which is each segment intersected with Δ-FSD (these intervals implicitly grow as we increase Δ).

For each pair (i, j) with $i < j$, h_i can Δ-*reach* h_j if there exists a monotone path from a point on h_i to a point on h_j inside Δ-FSD. Imagine Δ increasing from 0 to ∞. We consider four types of *events*:

- A *spawn event* for a horizontal segment h_i is the lowest Δ such that $h_i \neq \emptyset$.
- An *undertake* event for an index pair (i, j) is the lowest value Δ such that for all $\Delta' \geq \Delta$, the left endpoint of h_i is farther left than the left endpoint of h_j.
- A *join* event for an index pair (i, j) is the lowest value Δ for which the intervals h_i and h_j vertically overlap.

Storing a loser tree and a heap. Imagine increasing Δ from 0 to ∞. We compute the minimum number of join events to realise a monotone path from h_1 to h_k using a linked–list. We also a tree that we call a *loser tree* over h_1 to h_k. For each node of the loser tree stores a segment. Specifically, if a node has children storing segments h_i and h_j, we store h_i if in Δ-FSD the left endpoint of h_i is farther right to the left endpoint of h_j. We store in our heap the undertake event (i, j) — which may change the ordering between the left andpoints of h_i and h_j.

We also store a linked list of buckets. The bottom bucket $(h_1, h_2, \ldots h_i)$ contains all segments from h_1 up to the maximum index i such that there is a path from h_1 to h_i in Δ-FSD. The remaining items of each list store h_j for $j > i$ consecutively. Denote by ℓ the rightmost left endpoint of all segments in $(h_1, h_2, \ldots h_i)$ in Δ-FSD and let h_ℓ be the corresponding segment; note that we can obtain ℓ in $O(\log k)$ time from the loser tree. We store in our heap the spawn event for h_{i+1} and the join event between h_ℓ and h_{i+1}.

The argument. Suppose that for the current value of Δ, the bottom bucket is $(h_1, h_2, \ldots h_i)$. Then ℓ lies further right than the right endpoint of h_{i+1}. We pop the next event from the heap and set Δ to the corresponding value. We do a case analysis of which event is popped.

If the event is a spawn event of h_{i+1}, we test whether it becomes possible to go from h_1 to h_{i+1} by testing if the point that is h_{i+1} lies right of ℓ. If so, we

merge h_{i+1} into the bottom bucket. We recompute ℓ in $O(\log k)$ time using the loser tree, remove the old join event, and add the join event between the new h_ℓ and h_{i+2}.

If the event is an undertake event in the loser tree, then we update the loser tree. Note that these events change at most one node in the loser tree at a time, since at the time of an undertake event the left endpoints of the involved intervals coincide with one another. Thus, we update the loser tree in $O(\log k)$ time and we find the new value ℓ and the corresponding segment h_ℓ in $O(\log k)$ time. Since for pair (a, b), there can be only one undertake event and that invalidates all undertake events of b (since after an undertake, h_i grows faster than h_j), the loser tree generates at most $O(k \log k)$ undertake events and therefore this takes $O(k \log^2 k)$ total time.

If the event is the join event between h_ℓ and h_{i+1} then, given that it was not possible to go from h_1 to h_{i+1} before, it now becomes possible to go from h_1 to h_{i+1}. For the current updated Δ, the single vertical line where h_1 and h_{i+1} coincides corresponds to the monotonicity event between their corresponding points on σ and the corresponding edge of π. We add the corresponding monotonicity vertex to π. We then merge h_{i+1} into the bottom bucket. We recompute ℓ in $O(\log k)$ time using the loser tree. We add to the heap the join event between the new h_ℓ and h_{i+2} and the spawn event of h_{i+1}.

As a result, we compute in $O(k \log^2 k)$ time the minimum number of monotonicity vertices required for there to be a monotone path in the VE-graph from a to b. Note that the minimality comes from the fact that for any monotonicity event that we encounter this is the minimum value Δ that prevents further progression for a monotone curve. Since each path in the VE-graph has at most $O(n + m + M)$ vertices, applying this subroutine to all columns C and rows R where the path is non-monotone takes $O((n + m + M) \log^2(n + m + M))$ time:

Lemma 7. *Consider their VE-graph and let P be a min-cost path from $(1, 1)$ to (n, m). We can identify each row R and column C in which P is non-monotone. For each column C, where P is not x-monotone, let e be the corresponding edge of π. Let P, restricted to C, go from a to b.*

In $O((n + m + M) \log^2(n + m + M))$ total time we can, for all $(C, e, (a, b))$, compute the minimum set of monotonicity vertices V on e that need to be introduced such that in the resulting VE-graph, the minimum cost path from a to b is monotone.

Theorem 5. *Let (π, σ) be the original input curves with n and m vertices. Algorithm 1 can be supported by a data structure using $O(n + m + M)$ space, where $M \in O(n^3 + m^3)$ is the number of introduced monotonicity vertices. Each recursive call runs in $O(nm + M)$ time for Step 1 and $O((n + m + M) \log^2(n + m + M))$ time for Step 3. Under exact-value arithmetic, the procedure returns the exact Fréchet distance after $O(n^3 + m^3)$ recursive calls.*

Proof. Denote by (π, σ) the two curves that originally had n and m vertices, and that have M vertices added to them. Each iteration, we can compute the min-cost path from the source to the sink in the VE-graph in $O(n \cdot m + M)$ time using a simple linear scan from left to right over this vertex-weighted VE-graph. The resulting distance $\mathcal{V}_E(\pi, \sigma)$ is at most $\mathcal{D}_F(\pi, \sigma)$ and it is exactly $\mathcal{D}_F(\pi, \sigma)$ if there exists a path in the VE-graph realising $\mathcal{V}_E(\pi, \sigma)$ that is xy-monotone. A linear scan informs us that $\mathcal{V}_E(\pi, \sigma) = \Delta$. Given Δ, a depth-first search

over this graph informs us whether there exists a monotone path realising this distance in the VE-graph. Thus, each iteration, we can test in $O(n \cdot m + M)$ time whether our algorithm may terminate. If we do not terminate, we invoke the subroutine in $O((n + m + M) \log(n + m + M))$ time. It remains to show that our algorithm terminates within $O(n^3 + m^3)$ iterations.

Consider any pair of cells that share either a column, or a row. there are $O(nm^2 + mn^2)$ such pairs. Any path through the graph can be represented as a sequence of such pairs, such that two consecutive pairs do not share a row or column—essentially the maximal sub-rows and sub-columns traversed by the particular path. If the algorithm computes a path that is non-monotone, then every such pair is scrutinized, and a sequence of break points is introduced, such that these pairs may no longer be traversed non-monotonuously. In particular, any later path that shares a pair of cells in its represetntation must be monotonuous between this pair of cells. As there are only $O(nm^2 + mn^2)$ such pairs, and in every iteration at least one such pair is no longer non-monotonuously traverable, after a total of $O(nm^2 + mn^2)$ iterations the algorithm terminates, concluding the proof. □

B Lossless simplification

In this section we discuss details of our simplification, and how we obtain tighter lower bounds.

B.1 Hierarchical Simplification

We first describe the particular simplification we are using. We maintain a vertex-restricted simplification of the input, where on demand, we may subdivide any edge of the simplification introducing a vertex of the input in-between the two simplification vertices. Along with it, we maintain a joint traversal of every edge of the simplification with the corresponding subcurve of the input curve. This joint traversal is not necessarily optimal, but instead, we focus on a good, and quickly computable joint traversal.

To compute the joint traversal, let p_i and p_j be the vertices of π defining some edge of $\overline{\pi}$. For every p_k with $i \leq k \leq j$, we compute the value $t_k \in [0, 1]$ minimizing $\|p_k - (p_i + t_k(p_j - p_i))\|$, that is the point along the edge of the simplification minimizing the distance to p_k. From these, we obtain a matching between the edge of the simplification with the corresponding subcurve matching p_k with the point at $\max_{k' \leq k} t_{k'}$. It is easy to verify, that this is a 2-approximation of the optimal traversal. This traversal can be computed time that is linear in the number of vertices of the corresponding subcurve by a simple linear scan. In particular, this scan immediately yields what we define as a *split vertex*, i.e., a vertex realizing the maximum distance under this traversal.

B.2 A tighter lower bound

We next describe, how exactly we obtain our lower bound. For this, we endow every edge of the simplification with a negative weight corresponding to the negative maximum distance attained during the greedily computed joint traversal of the edge with its corresponding subcurve. This is the distance realised by the

split vertex of the edge. We compute the lower bound $LB(\overline{\pi}, \overline{\sigma})$ by essentially computing the Fréchet distance respecting the edge weights.

Definition 8 (Fréchet distance with additive weights)**.** *Let π and σ be two polygonal curves, where every edge of π and σ is endowed with a weight. Let $w_\pi : [0,1] \to \mathbb{R}$ be the function mapping any t to 0, if it coincides with a vertex of π, and the weight of the edge otherwise. Let w_σ be defined similarly. Then the Fréchet distance of π and σ with additive edge weights is defined as*

$$\mathcal{D}_F^w(\pi,\sigma) := \min_{\textit{traversals } (\alpha,\beta)} \max_{t\in[0,1]} \big(d\big(\pi(\alpha(t)), \sigma(\beta(t))\big) + w_\pi(\alpha(t)) + w_\sigma(\beta(t))\big) \ .$$

We first prove that this is indeed results in a lower bound to the Fréchet distance.

Lemma 9. *Let σ and π be two curves, and let $\overline{\sigma}$ and $\overline{\pi}$ be any vertex-restricted simplification of σ and π. Let w_σ and w_π be non-positive edge weights, such that the edge weight of any edge is at most the negative Fréchet distance of the edge and its corresponding subcurve. Then*

$$\mathcal{D}_F^w(\overline{\pi}, \overline{\sigma}) \leq \mathcal{D}_F(\pi,\sigma).$$

Proof. Consider first a traversal of every edge of $\overline{\pi}$ (resp. $\overline{\sigma}$) with its corresponding subcurve that matches the vertices of the edge to only the start and end point of the corresponding subcurve. The attained Fréchet distance w.r.t. these joint traversals coincides with the Fréchet distance in the limit. Let (α_π, β_π) (resp. $(\alpha_\sigma, \beta_\sigma)$) be this limit.

Consider a joint traversal (α, β) realizing $\mathcal{D}_F(\pi,\sigma)$. This traversal induces a joint traversal $(\overline{\alpha}, \overline{\beta})$ of $\overline{\pi}$ and $\overline{\sigma}$, where by triangle inequality we have that for all $t \in [0,1]$ it holds that

$$\|\pi(\alpha(t)) - \sigma(\beta(t))\| \geq \|\overline{\pi}(\overline{\alpha}(t)) - \overline{\sigma}(\overline{\beta}(t))\| \quad - \max_{s:\pi(s) \text{ matched with } \overline{\pi}(\overline{\alpha}(t)) \text{ by } (\alpha_\pi,\beta_\pi)} \|\pi(s) - \overline{\pi}(\overline{\alpha}(t))\| \\ - \max_{s:\sigma(s) \text{ matched with } \overline{\sigma}(\overline{\beta}(t)) \text{ by } (\alpha_\sigma,\beta_\sigma)} \|\sigma(s) - \overline{\sigma}(\overline{\beta}(t))\|.$$

But then in particular $\mathcal{D}_F(\pi,\sigma) \geq \mathcal{D}_F^w(\overline{\pi}, \overline{\sigma})$, as the edge weights are always less than these maxima. □

Observe that a traversal realizing the Fréchet distance with edge weights is also a traversal of $\overline{\sigma}$ and $\overline{\pi}$, and hence, together with joint traversals of π and $\overline{\pi}$, and σ and $\overline{\sigma}$ yield an upper bound to the Fréchet distance of π and σ. Finally, we need to show that the weighted Fréchet distance may be computed with the classical algorithm.

Lemma 10. *The Fréchet distance with additive weights can be computed by adding, for each edge e of the simplified curves, negative vertex weights to all eddies on that edges that correspond to the negative weight of the edge e.*

Proof. For this, it suffices to show, that monotonicity events for (π,σ) are in fact also monotonicity events for these additive edge weights. This is, however, immediately apparent, as the weights are not applied to the vertices of $\overline{\sigma}$ and $\overline{\pi}$, and hence the monotonicity events correspond exactly to the minima of the

distance function, as the only weight affecting a column (resp. row) is just a single edge weight, and hence constant. Thus, we can compute the Fréchet distance with additive weights by adding the weights to the eddies, and leaving the rest of the Fréchet distance computing algorithm unchanged. □

C Exact arithmetic

In this section we give more details on the exact arithmetic we perform. Ignoring the edge weights for now, every step of the algorithm can classically be performed by comparing square roots of fractions of integers, and fractions of integers, if the input coordinates are integers (or fractions of integers). These comparisons can be done exactly in $O(1)$ time, when provided the fractions inside the roots.

When introducing edge weights, we need to compare sums of square roots. The number of operations necessary to compare these expressions grows exponentially in the number of square roots in the expressions. To stay within practical realms, we do not do these types of comparisons. By Lemma 9, any overestimate of the edge weights also serves as a lower bound. Hence, for these lower bounds we generate an integer fraction that is larger than distance attained.

How to resolve square roots. It is common knowledge how to resolve constantly many square roots in an equation in an exact manner. The point we wish to make in this appendix section, is that whilst it is publicly known, it is also quite tedious and error-prone. Via our reduction of Lemma 9, which allows us to use integer over-estimations whenever an expression contains more than two square roots, we only are required to compare expressions of the form:

$$\frac{a}{b} \pm \sqrt{\frac{c}{d}} \leq \frac{e}{f} \pm \sqrt{\frac{g}{h}}.$$

These inequalities are resolved with exact integer arithmetic in the following way:

First, we note that we can relatively easily verify an expression of the form: $\frac{a}{b} \leq \pm\sqrt{\frac{c}{d}}$. Indeed, we can first check whether the individual fraction $\frac{a}{b}$ and $\frac{c}{d}$ are positive or negative. This results in a 4-case distinction, where in each case, we can square both sides to evaluate this expression exactly (adding a -1 whenever needed).

Once expressions of that form can be resolved, we can compare whether:

$$\sqrt{\frac{c}{d}} \leq \frac{e}{f} \pm \sqrt{\frac{g}{h}}.$$

Again, we can do case distinction based on whether $\frac{c}{d}$, $\frac{e}{f}$, $\frac{g}{h}$ are positive and negative. For each of the resulting eight cases, there is a way to square both sides such that the inequalities are preserved. The result is an expression with one fewer fraction, on which we recursively apply this strategy. Thus, via a sizeable but constant-depth decision tree, we can evaluate expressions of the form:

$$\frac{a}{b} \pm \sqrt{\frac{c}{d}} \leq \frac{e}{f} \pm \sqrt{\frac{g}{h}},$$

in an exact manner using integer arithmetic. We note that for certain input instances, such as the adversarial input derived from Orthogonal Vector instances, evaluating this decision tree is practically costly. Our implementation restricts the input to 32-bit integers (where we use 64-bit integers to resolve these square root equations). We note that it is possible to template the data type to allow for arbitrarily many input bits, but our implementation contains no such templates.

Table 5: Running time statistics (s). The best running time in bold, the second best in blue.

Data set	Max	Mean	StdDev	Median	80th %ile	90th %ile	95th %ile	99th %ile
				BKN				
geolife	**0.0767**	**0.0024**	0.0016	**0.0021**	**0.0027**	**0.0034**	**0.0042**	**0.0070**
pigeons	**0.1143**	**0.0040**	0.0022	**0.0035**	**0.0048**	**0.0066**	**0.0076**	**0.0102**
athens	**0.0075**	**0.0027**	0.0008	**0.0025**	**0.0033**	**0.0038**	**0.0043**	**0.0055**
OV	**0.0728**	**0.0208**	0.0206	**0.0111**	**0.0376**	**0.0580**	**0.0629**	**0.0682**
chicago	**0.0047**	**0.0018**	0.0002	**0.0018**	**0.0019**	**0.0020**	**0.0021**	**0.0024**
berlin	**0.0032**	0.0016	0.0002	0.0015	0.0017	0.0018	0.0018	0.0019
uniD	**0.0046**	**0.0018**	0.0002	**0.0018**	**0.0020**	**0.0021**	**0.0022**	**0.0024**
drifter	**0.0144**	**0.0024**	0.0009	**0.0022**	**0.0027**	**0.0032**	**0.0037**	**0.0060**
				Unleashed (accept_approx=true)				
geolife	250.5627	0.3367	3.5109	0.0074	0.0623	0.3244	0.9207	5.8376
pigeons	274.5574	0.7529	4.6066	0.0225	**0.1623**	0.9021	3.7495	13.2745
athens	29.4879	0.1754	1.1123	0.0195	**0.1081**	0.2415	0.4817	2.4792
OV	24.8995	1.2463	3.2985	0.0609	1.3038	3.8347	5.5689	15.1980
chicago	0.5972	**0.0035**	0.0144	**0.0023**	**0.0040**	0.0054	0.0064	0.0093
berlin	0.1214	**0.0001**	0.0018	**0.0001**	**0.0001**	**0.0001**	**0.0002**	**0.0002**
uniD	**0.1239**	0.0024	0.0045	0.0020	0.0029	0.0035	0.0042	0.0086
drifter	60.7184	0.1177	1.1667	0.0095	**0.0529**	0.1730	0.4747	1.8644
				Unleashed (accept_approx=false)				
geolife	233.6160	0.3673	3.7389	**0.0069**	**0.0621**	0.3263	1.0108	6.4489
pigeons	>300	0.8508	9.0841	**0.0222**	0.1645	0.9395	3.7772	13.5008
athens	29.1741	0.2133	1.3476	**0.0190**	0.1091	0.2808	0.5125	3.6473
OV	**24.6879**	**1.2132**	3.2387	**0.0580**	**1.1886**	**3.7313**	**5.2031**	**15.0121**
chicago	0.4679	0.0036	0.0147	0.0024	0.0040	**0.0053**	**0.0063**	**0.0088**
berlin	0.0059	**0.0001**	0.0001	**0.0001**	**0.0001**	**0.0001**	**0.0001**	**0.0002**
uniD	0.1402	**0.0023**	0.0048	**0.0019**	**0.0027**	**0.0032**	**0.0041**	**0.0076**
drifter	61.2818	0.1199	1.1781	**0.0091**	0.0555	0.1919	0.4474	1.9376
				Exact (with_dijkstra=true)				
geolife	**7.6259**	0.1028	0.1874	0.0525	0.1381	0.2414	0.3633	0.7742
pigeons	3.1337	0.1672	0.1563	0.1268	0.2322	0.3201	0.4239	0.7432
athens	5.3387	0.1975	0.3426	0.0912	0.2434	0.4529	0.7634	1.6910
OV	24.4039	3.3321	5.0168	0.7700	6.7386	10.2982	14.0325	19.9612
chicago	0.0543	0.0076	0.0076	0.0043	0.0122	0.0182	0.0241	0.0344
berlin	0.0037	**0.0004**	0.0002	**0.0003**	**0.0004**	**0.0004**	0.0006	**0.0014**
uniD	0.2445	0.0077	0.0098	0.0064	0.0095	0.0114	0.0138	0.0271
drifter	1.7855	0.0806	0.1061	0.0519	0.1182	0.1748	0.2428	0.4936
				Exact (with_dijkstra=false)				
geolife	17.5543	**0.0984**	0.1903	0.0505	0.1330	**0.2312**	**0.3468**	**0.7444**
pigeons	**2.9708**	**0.1649**	0.1472	0.1265	0.2305	**0.3153**	**0.4130**	**0.7262**
athens	**2.6424**	**0.1617**	0.2359	0.0851	0.2136	**0.3816**	**0.5906**	**1.2356**
OV	28.2322	4.5376	6.0343	1.5410	8.5852	13.5812	16.9130	23.8446
chicago	**0.0522**	0.0075	0.0073	0.0043	0.0124	0.0179	0.0231	0.0326
berlin	**0.0029**	**0.0004**	0.0002	**0.0003**	**0.0004**	0.0005	0.0007	**0.0014**
uniD	0.2809	0.0077	0.0102	0.0064	0.0096	0.0114	0.0139	0.0266
drifter	**1.6604**	**0.0743**	0.0889	0.0505	0.1123	**0.1609**	**0.2215**	**0.4069**